

- The “published version” of the JSON Schema validation specification is considered production-ready, but not strictly speaking “stable.” When using JSON Schema validation, ensure there are no gaps with the guidance in the requirements below.
- Any JSON Schema validation libraries in use should also be monitored and updated if necessary once the standard is formalized.
- DTD validation should not be used, and framework DTD evaluation should be disabled, to avoid issues with XXE attacks against DTDs.

#	Description	Level
2.2.1	Verify that input is validated to enforce business or functional expectations for that input. This should either use positive validation against an allow list of values, patterns, and ranges, or be based on comparing the input to an expected structure and logical limits according to predefined rules. For L1, this can focus on input which is used to make specific business or security decisions. For L2 and up, this should apply to all input.	1
2.2.2	Verify that the application is designed to enforce input validation at a trusted service layer. While client-side validation improves usability and should be encouraged, it must not be relied upon as a security control.	1
2.2.3	Verify that the application ensures that combinations of related data items are reasonable according to the pre-defined rules.	2

V2.3 Business Logic Security

This section considers key requirements to ensure that the application enforces business logic processes in the correct way and is not vulnerable to attacks that exploit the logic and flow of the application.

#	Description	Level
2.3.1	Verify that the application will only process business logic flows for the same user in the expected sequential step order and without skipping steps.	1
2.3.2	Verify that business logic limits are implemented per the application’s documentation to avoid business logic flaws being exploited.	2
2.3.3	Verify that transactions are being used at the business logic level such that either a business logic operation succeeds in its entirety or it is rolled back to the previous correct state.	2

#	Description	Level
2.3.4	Verify that business logic level locking mechanisms are used to ensure that limited quantity resources (such as theater seats or delivery slots) cannot be double-booked by manipulating the application's logic.	2
2.3.5	Verify that high-value business logic flows require multi-user approval to prevent unauthorized or accidental actions. This could include but is not limited to large monetary transfers, contract approvals, access to classified information, or safety overrides in manufacturing.	3

V2.4 Anti-automation

This section includes anti-automation controls to ensure that human-like interactions are required and excessive automated requests are prevented.

#	Description	Level
2.4.1	Verify that anti-automation controls are in place to protect against excessive calls to application functions that could lead to data exfiltration, garbage-data creation, quota exhaustion, rate-limit breaches, denial-of-service, or overuse of costly resources.	2
2.4.2	Verify that business logic flows require realistic human timing, preventing excessively rapid transaction submissions.	3

References

For more information, see also:

- OWASP Web Security Testing Guide: Input Validation Testing
- OWASP Web Security Testing Guide: Business Logic Testing
- Anti-automation can be achieved in many ways, including the use of the OWASP Automated Threats to Web Applications
- OWASP Input Validation Cheat Sheet
- JSON Schema

V3 Web Frontend Security

Control Objective

This category focuses on requirements designed to protect against attacks executed via a web frontend. These requirements do not apply to machine-to-machine solutions.

V3.1 Web Frontend Security Documentation

This section outlines the browser security features that should be specified in the application's documentation.

#	Description	Level
3.1.1	Verify that application documentation states the expected security features that browsers using the application must support (such as HTTPS, HTTP Strict Transport Security (HSTS), Content Security Policy (CSP), and other relevant HTTP security mechanisms). It must also define how the application must behave when some of these features are not available (such as warning the user or blocking access).	3

V3.2 Unintended Content Interpretation

Rendering content or functionality in an incorrect context can result in malicious content being executed or displayed.

#	Description	Level
3.2.1	Verify that security controls are in place to prevent browsers from rendering content or functionality in HTTP responses in an incorrect context (e.g., when an API, a user-uploaded file or other resource is requested directly). Possible controls could include: not serving the content unless HTTP request header fields (such as Sec-Fetch-*) indicate it is the correct context, using the sandbox directive of the Content-Security-Policy header field or using the attachment disposition type in the Content-Disposition header field.	1
3.2.2	Verify that content intended to be displayed as text, rather than rendered as HTML, is handled using safe rendering functions (such as <code>createTextNode</code> or <code>textContent</code>) to prevent unintended execution of content such as HTML or JavaScript.	1

#	Description	Level
3.2.3	Verify that the application avoids DOM clobbering when using client-side JavaScript by employing explicit variable declarations, performing strict type checking, avoiding storing global variables on the document object, and implementing namespace isolation.	3

V3.3 Cookie Setup

This section outlines requirements for securely configuring sensitive cookies to provide a higher level of assurance that they were created by the application itself and to prevent their contents from leaking or being inappropriately modified.

#	Description	Level
3.3.1	Verify that cookies have the 'Secure' attribute set, and if the '__Host-' prefix is not used for the cookie name, the '__Secure-' prefix must be used for the cookie name.	1
3.3.2	Verify that each cookie's 'SameSite' attribute value is set according to the purpose of the cookie, to limit exposure to user interface redress attacks and browser-based request forgery attacks, commonly known as cross-site request forgery (CSRF).	2
3.3.3	Verify that cookies have the '__Host-' prefix for the cookie name unless they are explicitly designed to be shared with other hosts.	2
3.3.4	Verify that if the value of a cookie is not meant to be accessible to client-side scripts (such as a session token), the cookie must have the 'HttpOnly' attribute set and the same value (e. g. session token) must only be transferred to the client via the 'Set-Cookie' header field.	2
3.3.5	Verify that when the application writes a cookie, the cookie name and value length combined are not over 4096 bytes. Overly large cookies will not be stored by the browser and therefore not sent with requests, preventing the user from using application functionality which relies on that cookie.	3

V3.4 Browser Security Mechanism Headers

This section describes which security headers should be set on HTTP responses to enable browser security features and restrictions when handling responses from the application.

#	Description	Level
3.4.1	Verify that a Strict-Transport-Security header field is included on all responses to enforce an HTTP Strict Transport Security (HSTS) policy. A maximum age of at least 1 year must be defined, and for L2 and up, the policy must apply to all subdomains as well.	1
3.4.2	Verify that the Cross-Origin Resource Sharing (CORS) Access-Control-Allow-Origin header field is a fixed value by the application, or if the Origin HTTP request header field value is used, it is validated against an allowlist of trusted origins. When 'Access-Control-Allow-Origin: *' needs to be used, verify that the response does not include any sensitive information.	1
3.4.3	Verify that HTTP responses include a Content-Security-Policy response header field which defines directives to ensure the browser only loads and executes trusted content or resources, in order to limit execution of malicious JavaScript. As a minimum, a global policy must be used which includes the directives object-src 'none' and base-uri 'none' and defines either an allowlist or uses nonces or hashes. For an L3 application, a per-response policy with nonces or hashes must be defined.	2
3.4.4	Verify that all HTTP responses contain an 'X-Content-Type-Options: nosniff' header field. This instructs browsers not to use content sniffing and MIME type guessing for the given response, and to require the response's Content-Type header field value to match the destination resource. For example, the response to a request for a style is only accepted if the response's Content-Type is 'text/css'. This also enables the use of the Cross-Origin Read Blocking (CORB) functionality by the browser.	2
3.4.5	Verify that the application sets a referrer policy to prevent leakage of technically sensitive data to third-party services via the 'Referer' HTTP request header field. This can be done using the Referrer-Policy HTTP response header field or via HTML element attributes. Sensitive data could include path and query data in the URL, and for internal non-public applications also the hostname.	2
3.4.6	Verify that the web application uses the frame-ancestors directive of the Content-Security-Policy header field for every HTTP response to ensure that it cannot be embedded by default and that embedding of specific resources is allowed only when necessary. Note that the X-Frame-Options header field, although supported by browsers, is obsolete and may not be relied upon.	2
3.4.7	Verify that the Content-Security-Policy header field specifies a location to report violations.	3

#	Description	Level
3.4.8	Verify that all HTTP responses that initiate a document rendering (such as responses with Content-Type text/html), include the Cross-Origin-Opener-Policy header field with the same-origin directive or the same-origin-allow-popups directive as required. This prevents attacks that abuse shared access to Window objects, such as tabnabbing and frame counting.	3

V3.5 Browser Origin Separation

When accepting a request to sensitive functionality on the server side, the application needs to ensure the request is initiated by the application itself or by a trusted party and has not been forged by an attacker.

Sensitive functionality in this context could include accepting form posts for authenticated and non-authenticated users (such as an authentication request), state-changing operations, or resource-demanding functionality (such as data export).

The key protections here are browser security policies like Same Origin Policy for JavaScript and also SameSite logic for cookies. Another common protection is the CORS preflight mechanism. This mechanism will be critical for endpoints designed to be called from a different origin, but it can also be a useful request forgery prevention mechanism for endpoints which are not designed to be called from a different origin.

#	Description	Level
3.5.1	Verify that, if the application does not rely on the CORS preflight mechanism to prevent disallowed cross-origin requests to use sensitive functionality, these requests are validated to ensure they originate from the application itself. This may be done by using and validating anti-forgery tokens or requiring extra HTTP header fields that are not CORS-safelisted request-header fields. This is to defend against browser-based request forgery attacks, commonly known as cross-site request forgery (CSRF).	1
3.5.2	Verify that, if the application relies on the CORS preflight mechanism to prevent disallowed cross-origin use of sensitive functionality, it is not possible to call the functionality with a request which does not trigger a CORS-preflight request. This may require checking the values of the 'Origin' and 'Content-Type' request header fields or using an extra header field that is not a CORS-safelisted header-field.	1

#	Description	Level
3.5.3	Verify that HTTP requests to sensitive functionality use appropriate HTTP methods such as POST, PUT, PATCH, or DELETE, and not methods defined by the HTTP specification as “safe” such as HEAD, OPTIONS, or GET. Alternatively, strict validation of the Sec-Fetch-* request header fields can be used to ensure that the request did not originate from an inappropriate cross-origin call, a navigation request, or a resource load (such as an image source) where this is not expected.	1
3.5.4	Verify that separate applications are hosted on different hostnames to leverage the restrictions provided by same-origin policy, including how documents or scripts loaded by one origin can interact with resources from another origin and hostname-based restrictions on cookies.	2
3.5.5	Verify that messages received by the postMessage interface are discarded if the origin of the message is not trusted, or if the syntax of the message is invalid.	2
3.5.6	Verify that JSONP functionality is not enabled anywhere across the application to avoid Cross-Site Script Inclusion (XSSI) attacks.	3
3.5.7	Verify that data requiring authorization is not included in script resource responses, like JavaScript files, to prevent Cross-Site Script Inclusion (XSSI) attacks.	3
3.5.8	Verify that authenticated resources (such as images, videos, scripts, and other documents) can be loaded or embedded on behalf of the user only when intended. This can be accomplished by strict validation of the Sec-Fetch-* HTTP request header fields to ensure that the request did not originate from an inappropriate cross-origin call, or by setting a restrictive Cross-Origin-Resource-Policy HTTP response header field to instruct the browser to block returned content.	3

V3.6 External Resource Integrity

This section provides guidance for the safe hosting of content on third-party sites.

#	Description	Level
3.6.1	Verify that client-side assets, such as JavaScript libraries, CSS, or web fonts, are only hosted externally (e.g., on a Content Delivery Network) if the resource is static and versioned and Subresource Integrity (SRI) is used to validate the integrity of the asset. If this is not possible, there should be a documented security decision to justify this for each resource.	3

V3.7 Other Browser Security Considerations

This section includes various other security controls and modern browser security features required for client-side browser security.

#	Description	Level
3.7.1	Verify that the application only uses client-side technologies which are still supported and considered secure. Examples of technologies which do not meet this requirement include NSAPI plugins, Flash, Shockwave, ActiveX, Silverlight, NACL, or client-side Java applets.	2
3.7.2	Verify that the application will only automatically redirect the user to a different hostname or domain (which is not controlled by the application) where the destination appears on an allowlist.	2
3.7.3	Verify that the application shows a notification when the user is being redirected to a URL outside of the application's control, with an option to cancel the navigation.	3
3.7.4	Verify that the application's top-level domain (e.g., site.tld) is added to the public preload list for HTTP Strict Transport Security (HSTS). This ensures that the use of TLS for the application is built directly into the main browsers, rather than relying only on the Strict-Transport-Security response header field.	3
3.7.5	Verify that the application behaves as documented (such as warning the user or blocking access) if the browser used to access the application does not support the expected security features.	3

References

For more information, see also:

- Set-Cookie __Host- prefix details

- OWASP Content Security Policy Cheat Sheet
- OWASP Secure Headers Project
- OWASP Cross-Site Request Forgery Prevention Cheat Sheet
- HSTS Browser Preload List submission form
- OWASP DOM Clobbering Prevention Cheat Sheet

V4 API and Web Service

Control Objective

Several considerations apply specifically to applications that expose APIs for use by web browsers or other consumers (commonly using JSON, XML, or GraphQL). This chapter covers the relevant security configurations and mechanisms that should be applied.

Note that authentication, session management, and input validation concerns from other chapters also apply to APIs, so this chapter cannot be taken out of context or tested in isolation.

V4.1 Generic Web Service Security

This section addresses general web service security considerations and, consequently, basic web service hygiene practices.

#	Description	Level
4.1.1	Verify that every HTTP response with a message body contains a Content-Type header field that matches the actual content of the response, including the charset parameter to specify safe character encoding (e.g., UTF-8, ISO-8859-1) according to IANA Media Types, such as “text/”, “/+xml” and “/xml”.	1
4.1.2	Verify that only user-facing endpoints (intended for manual web-browser access) automatically redirect from HTTP to HTTPS, while other services or endpoints do not implement transparent redirects. This is to avoid a situation where a client is erroneously sending unencrypted HTTP requests, but since the requests are being automatically redirected to HTTPS, the leakage of sensitive data goes undiscovered.	2
4.1.3	Verify that any HTTP header field used by the application and set by an intermediary layer, such as a load balancer, a web proxy, or a backend-for-frontend service, cannot be overridden by the end-user. Example headers might include X-Real-IP, X-Forwarded-*, or X-User-ID.	2

#	Description	Level
4.1.4	Verify that only HTTP methods that are explicitly supported by the application or its API (including OPTIONS during preflight requests) can be used and that unused methods are blocked.	3
4.1.5	Verify that per-message digital signatures are used to provide additional assurance on top of transport protections for requests or transactions which are highly sensitive or which traverse a number of systems.	3

V4.2 HTTP Message Structure Validation

This section explains how the structure and header fields of an HTTP message should be validated to prevent attacks such as request smuggling, response splitting, header injection, and denial of service via overly long HTTP messages.

These requirements are relevant for general HTTP message processing and generation, but are especially important when converting HTTP messages between different HTTP versions.

#	Description	Level
4.2.1	Verify that all application components (including load balancers, firewalls, and application servers) determine boundaries of incoming HTTP messages using the appropriate mechanism for the HTTP version to prevent HTTP request smuggling. In HTTP/1.x, if a Transfer-Encoding header field is present, the Content-Length header must be ignored per RFC 2616. When using HTTP/2 or HTTP/3, if a Content-Length header field is present, the receiver must ensure that it is consistent with the length of the DATA frames.	2
4.2.2	Verify that when generating HTTP messages, the Content-Length header field does not conflict with the length of the content as determined by the framing of the HTTP protocol, in order to prevent request smuggling attacks.	3
4.2.3	Verify that the application does not send nor accept HTTP/2 or HTTP/3 messages with connection-specific header fields such as Transfer-Encoding to prevent response splitting and header injection attacks.	3
4.2.4	Verify that the application only accepts HTTP/2 and HTTP/3 requests where the header fields and values do not contain any CR (\r), LF (\n), or CRLF (\r\n) sequences, to prevent header injection attacks.	3